

Special Participation D - Refactoring HW 6 Coding Portions

Author: Hanyanhg Gu (SID: 3038838194), Zimu Wang (SID: 3038960121)

Executive Summary

This report details the comprehensive refactoring of the `hw06_code` repository, transforming a set of educational Jupyter notebooks into a cohesive, engineering-focused codebase. The primary objective was to bridge the gap between academic "script-style" deep learning code and professional software engineering standards, without compromising the pedagogical value of the assignments.

The refactoring process was strictly guided by three core pillars: **PEP 8 compliance**, the **Single Responsibility Principle (SRP)**, and the **Google ML Engineering Guide**.

First, to adhere to the **Single Responsibility Principle**, we dismantled the monolithic structure of the original notebooks. Code that defined model architectures, training loops, and utility functions was extracted from the notebooks and organized into dedicated Python modules (`architectures.py` , `trainer.py` , `utils.py` , `profiling.py` , `optimizers.py` , `graph_utils.py`). This separation ensures that notebooks are reserved for high-level experimentation and narrative, while the heavy lifting is handled by robust, testable modules.

Second, we enforced **PEP 8** standards across the board. This involved standardizing naming conventions (e.g., using `snake_case` for functions and `CamelCase` for classes), adding comprehensive type hints to function signatures, and including docstrings. These changes significantly improve code readability and maintainability, making it easier for students to understand the interfaces they are working with.

Finally, drawing from the **Google ML Engineering Guide**, we implemented best practices for reproducibility and configuration management. Global variables, which often lead to silent errors and reproducibility issues, were replaced with structured `dataclasses` (e.g., `TrainingConfig`). We also introduced a centralized random seed utility to ensure that experiments are deterministic across different runs and hardware setups.

The result is a suite of notebooks that function identically to their predecessors but serve as examples of clean, production-ready machine learning code.

Detailed Notebook Refactoring

Part 1. wandb.ipynb & tensorboard.ipynb

Refactoring Approach:

The monolithic training loops and global variable configurations were replaced with a modular `Trainer` class and a `TrainingConfig` dataclass. Redundant model definitions (like `Net`) were removed in favor of a centralized `BasicConvNet` in `architectures.py`. The logging logic was abstracted to support both WandB and TensorBoard.

LLM Prompt:

"Refactor `wandb.ipynb` to use a `Trainer` class. Move the training loop logic into a separate python file `trainer.py`. The `Trainer` should accept a `logger` argument that abstracts the difference between WandB and TensorBoard. Update the notebook to import this class and run an experiment using a configuration object."

Original Code Structure:

```
# Global variables
learning_rate = 0.01
epochs = 2
# ...

# Redundant model definition
class Net(nn.Module):
    # ...

# Monolithic training loop
for epoch in range(epochs):
    for i, data in enumerate(trainloader, 0):
        # ... training logic ...
        if i % log_freq == 0:
            wandb.log(...)
```

New Structure:

```

from trainer import Trainer, TrainingConfig
from architectures import BasicConvNet

# Configuration Object
config = TrainingConfig(
    learning_rate=0.01,
    epochs=2,
    # ...
)

# Modular Trainer
trainer = Trainer(
    model=BasicConvNet(),
    train_loader=trainloader,
    test_loader=testloader,
    config=config,
    device=device,
    logger=wandb # or writer for TensorBoard
)

trainer.train()

```

Intuition & Educational Value:

By decoupling the training loop from the logging mechanism, students can see that the core learning algorithm (SGD loop) is independent of the tool used to observe it. This abstraction allows them to easily switch between WandB and TensorBoard without rewriting the training logic, reinforcing the concept of modularity. It also highlights how configuration objects make experiments reproducible, a key concept in scientific deep learning. Ideally, this setup allows a student to simply swap the `logger` object to see how different tools visualize the *exact same* training process, isolating the variable of interest.

Part 2. GPUMemory.ipynb

Refactoring Approach:

Ad-hoc subprocess calls to `nvidia-smi` and manual print statements were replaced with a reusable `MemoryTracker` context manager in `profiling.py`. This ensures consistent logging and cleaner notebook cells.

LLM Prompt:

"Create a `profiling.py` module. Implement a `MemoryTracker` context manager that logs GPU memory usage. Refactor `GPUMemory.ipynb` to use this context manager for its experiments instead of the ad-hoc logging code. Ensure it handles cases where CUDA is not available gracefully."

Original Code Structure:

```
def run_nvidia_smi():
    if torch.cuda.is_available():
        print(subprocess.check_output("nvidia-smi", shell=True).decode("utf-8"))

# Usage scattered throughout the notebook
run_nvidia_smi()
# ... code block ...
run_nvidia_smi()
```

New Structure:

```
from profiling import MemoryTracker

# Context Manager handles logging automatically
with MemoryTracker(device) as tracker:
    # ... code block ...
    # Memory usage change is printed automatically on exit
```

Intuition & Educational Value:

Using a context manager (`with MemoryTracker...`) teaches students about resource management lifecycles. Instead of cluttering their mental model with repeated boilerplate code, they can focus on the *cause and effect* of their operations (e.g., "What happens to memory when I increase batch size?"). It abstracts the "how" of measurement so they can focus on the "what" and "why" of the results. This pattern is ubiquitous in Python (e.g., opening files), and applying it here reinforces Pythonic thinking in an ML context.

Part 3. `q_coding_muon.ipynb`

Refactoring Approach:

The standalone `newton_schulz_orthogonalize` function and manual parameter update loops were encapsulated into a proper PyTorch `Optimizer` class named `Muon` in `optimizers.py`. The model definition was moved to `architectures.py`.

LLM Prompt:

"Refactor the Muon implementation in `q_coding_muon.ipynb` . Instead of a standalone function and manual updates, implement a `Muon` class that inherits from `torch.optim.Optimizer` . Move this class to `optimizers.py` . Update the notebook to use this standard PyTorch optimizer interface."

Original Code Structure:

```
# Standalone function
def newton_schulz_orthogonalize(X, num_iters):
    # ...

# Manual optimization loop
for p in model.parameters():
    # ... manual gradient updates ...
    p.data.add_(...)
```

New Structure:

```
from optimizers import Muon

# Standard PyTorch Optimizer interface
optimizer = Muon(model.parameters(), lr=0.02, momentum=0.9)

# Standard training step
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Intuition & Educational Value:

Refactoring the Muon optimizer into a standard PyTorch `Optimizer` class is crucial for intuition. It shows students that custom optimizers are not "magic scripts" but structured components that interact with the model via a standard interface (`step()` , `zero_grad()`). This empowers them to plug in their custom optimizer into *any* standard PyTorch training loop, facilitating easy comparison with SGD or Adam, just like in a real research setting. It transforms the assignment from "implementing a math formula" to "building a reusable ML tool." (E.g. for Special Participation D).

Part 4. q_zkc.ipynb

Refactoring Approach:

Scattered helper functions for graph processing were moved to `graph_utils.py`. The Graph Neural Network layer, originally a standalone class with manual forward/backward methods, was refactored into a standard `nn.Module` in `architectures.py`, while preserving the educational "fill-in-the-blank" structure.

LLM Prompt:

```
"Refactor q_zkc.ipynb to structure the GNN implementation as a PyTorch nn.Module . Create a GCNLayer class that encapsulates the math currently done in the global scope. Move the data loading and graph helper functions to graph_utils.py ."
```

Original Code Structure:

```
# Inline helper functions
def get_graph_metadata(graph): ...
def get_xavier_init(input_dim, output_dim): ...

# Non-standard class
class GNN_Layer():
    def forward_pass(self, A, X):
        # ...
```

New Structure:

```
from architectures import GCNLayer
from graph_utils import get_graph_metadata, get_xavier_init

# Standard nn.Module
class GCNLayer(nn.Module):
    def forward(self, A, X):
        # ... standard forward pass ...
```

Intuition & Educational Value:

Moving the GNN layer to a `nn.Module` bridges the gap between mathematical theory and framework implementation. Students often struggle to see how matrix multiplications map to "layers". By subclassing `nn.Module`, they see that a GNN layer is just another building block with state (weights) and a forward pass, exactly like a `Linear` or `Conv2d` layer. This demystifies GNNs and shows they

are first-class citizens in the deep learning ecosystem, encouraging students to compose them into larger architectures.